

Oil Price Prediction via Financial News Scraping

1 Motivation

Oil prices are among the most consequential and volatile signals in the global economy. Recent geopolitical tensions, including the ongoing Strait of Hormuz conflict and escalating U.S.–China tariffs, have produced intraday Brent crude swings exceeding 10%. These severe fluctuations place significant economic strain on working-class demographics. Because market-moving headlines often emerge hours or days before they are reflected in formal economic releases, the high-frequency collection of financial news offers a crucial leading indicator that purely price-based models cannot capture. To harness this predictive power, our pipeline harvests over **1,000 relevant news articles** directly from Google News, linking each article to the trading-day oil price it likely influenced.

2 Query Design and Collection Schedule

We authored **27 Boolean-style queries** (Table 1) that span oil & energy markets, macroeconomic policy, geopolitics, and major equities. A scheduler submitted every query to Google News once per day from **April 28 through May 12, 2026**, accumulating **220 unique search-result pages** with an average of ≈ 5 article references each—yielding roughly **1,000 article URLs** in total across 14 date-partitioned batches (one `tasks_YYYY-MM-DD.json` per day).

Table 1: Query categories and representative examples

Category	Example Query
Oil & Energy	oil price OR crude oil OR energy market news
Macro / Fed	Federal Reserve OR Fed rate OR interest rate hike OR cut
Geopolitics	China economy OR US-China trade war OR tariffs
Major Equities	NVDA OR Nvidia stock OR AI OR semiconductor
Crypto / Fintech	crypto OR bitcoin OR cryptocurrency regulation
Earnings	earnings season OR corporate earnings report

3 Anti-Detection Scraping Pipeline

Google aggressively rate-limits and blocks automated clients. Our stack layers four independent defenses so that every request appears indistinguishable from an organic home user browsing the web.

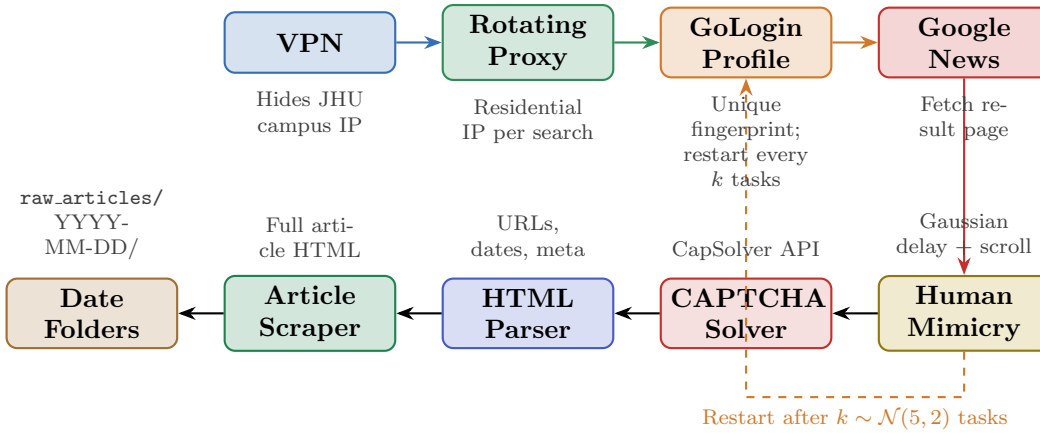


Figure 1: End-to-end data collection pipeline. Network identity flows left-to-right (top row); scraped data flows right-to-left through processing stages (bottom row). The dashed arc marks the GoLogin profile restart cycle.

VPN. All traffic originates from a VPN endpoint, masking the Johns Hopkins University campus IP block. Google applies aggressive rate-limiting to institutional subnets; routing outbound connections through the VPN is the first and most fundamental layer of anonymisation.

Rotating Residential Proxy. Every outbound request is further tunnelled through a residential proxy pool that assigns a *fresh IP address for each individual Google search*. Unlike datacenter IPs—which Google’s heuristics flag within seconds—residential addresses are owned by real ISP subscribers and are essentially indistinguishable from authentic home traffic.

GoLogin Anti-Detect Browser. GoLogin provisions isolated Chromium browser profiles, each carrying a unique *browser fingerprint*: user-agent string, screen resolution, WebGL renderer hash, installed-font set, timezone, and canvas noise. The scraper boots a profile via the GoLogin SDK, then Playwright connects to it over the Chrome DevTools Protocol (CDP) and executes the search. After every $k \sim \mathcal{N}(5, 2)$ tasks—a Gaussian-randomised batch size—the profile is *stopped and restarted*, flushing all session memory, cookies, and cached state. This severs any continuity Google might use to link successive queries to the same browser identity.

Human Behaviour Mimicry. Two layers of behavioural camouflage are applied on every page load. First, inter-request pauses follow a Gaussian distribution $\Delta t \sim \mathcal{N}(\frac{t_{\min} + t_{\max}}{2}, \frac{t_{\max} - t_{\min}}{4})$ rather than a suspicious uniform interval. Second, 3–5 erratic mouse-wheel scroll events are injected after each page loads, with a 30% probability of reversing direction by 150 px to simulate reading and backtracking—patterns that exactly match real user engagement.

CAPTCHA Solver. If Google redirects to its `sorry/index` challenge or a `.g-recaptcha` widget appears, the engine automatically extracts the `data-sitekey`, submits a `ReCaptchaV2TaskProxyless` task to the **CapSolver** API, polls for the solved token, injects it into the hidden `g-recaptcha-response` field, and programmatically submits the form—fully unattended, with no human intervention required.

4 Parsing and Article Retrieval

`parser.py` processes each saved Google News HTML file through four complementary extraction strategies applied in priority order: (1) JSON-LD structured data (highest fidelity), (2) OpenGraph / `<meta>` page-level tags, (3) Google News card DOM selectors (`div.SoaBEf`, `g-card`), and (4) generic CMS card heuristics (`.post`, `.story`, `.card`). Duplicates keyed by URL are merged across strategies, ensuring no article is double-counted. Each extracted record carries the **title**, **URL**, **source domain**, **snippet**, **publication date**, **originating query**, and **search date**—written as one JSON file per Google search

HTML.

`new_article_scraper.py` reads the full parsed-JSON corpus, de-duplicates URLs across all dates, and fetches the complete HTML of each article using a stealth Playwright session. Saved files are placed in **date-partitioned subdirectories** (`raw_articles/2026-05-09/`), aligning each article to the trading-day oil price it may have influenced.

Table 2: Dataset collection summary

Metric	Value
Collection period	April 25 – May 12, 2026
Unique queries	27
Google search result pages	220
Avg. articles per page	≈ 5
Total article URLs	$\approx 1,000$
Date partitions	14 trading days

5 Predictive Model: Learning from the News

Our goal was to build a model that reads a week of news and predicts if oil prices will go up or down. While modern AI often uses massive, complex networks, our journey taught us that for niche financial data, simpler is often smarter.



Figure 2: Model Architecture: From raw HTML to price prediction.

5.1 Design Philosophy: Why Smaller Worked Better

Our first attempts (v1 and v2) used large, complex transformer models. However, these models were too powerful for our dataset; they memorized the noise instead of learning the trends, leading to a poor 15% accuracy on new data. For our final model (v3), we switched to a BiLSTM architecture (Figure 2). This model is specifically designed to understand sequences in time without getting lost in the details.

5.2 The Sliding Window and Data Sparsity

To simulate a real-world trader, we used a sliding window strategy. The model evaluates 5 days of context (Monday–Friday) to predict the market’s direction on the 6th day (the following Monday).

Data sparsity presented a major structural challenge. Financial news is inherently uneven in its distribution: while some days yielded over 20 relevant articles, weekends and holidays often produced none. This sparsity made it difficult for the model to identify consistent temporal patterns, as it frequently had to base its predictions on minimal information within its 5-day sliding window. Furthermore, our data collection period was restricted to just 17 days due to the time required to develop and deploy the scraper. We hypothesize that with an extended timeframe to gather a larger corpus of news, our transformer models would be significantly less prone to overfitting, enabling them to capture more robust, generalized signals across the broader oil market.

5.3 How the Model ”Reads” and ”Decides”

The model is built from five components:

- We first use *FinBERT* to convert news articles into mathematical sentiment scores. This is our foundation because it gives the computer a sense of the market as a high dimensional vector. Also note that this saves us a lot of trouble dealing with stop words and word sense disambiguation.
- Since the raw scores can be messy and high-dimensional (776 dimension). We use a projection layer to filter this data and compress it down to the most important features (256 dimension). This prevents the model from getting overwhelmed by unimportant noise in the text data.
- Markets have a memory. Our bidirectional LSTM reads the 5-day sequence forward and backward. This allows it to understand how news on Monday might be reinterpreted by a headline on Friday and capturing the full story of the week.
- Since not every day is a news day. The attention mechanism assigns a weight to each day. This would allow it to be If Friday had a major policy shift, the model weights Friday more and ignores the noise from quieter days.
- Finally, the model takes the focused story and maps it to a final choice: Up or Down.

6 Measuring Success

To test the model fairly, we used a chronological split (80% training, 10% validation, 10% test). We trained on the past and tested on the future to ensure the model never cheated by seeing tomorrow’s headlines.

Before detailing the final metrics, it is important to highlight a major problem we encountered: the model’s accuracy was highly volatile. We observed that performance depended heavily on the exact cutoff date used for the train-test split. Shifting the chronological boundary by even a single day could cause the results to swing significantly. We attribute this instability to not having enough data. Because our collection window was only 14 days, the test set often contained unique market conditions that the model had not experienced during training. This scarcity of data is a major limitation of the current pipeline.

Despite this volatility, we evaluated our model in two modes to test its baseline capabilities:

Binary vs. 3-Way Classification.

- Binary (Up vs. Down): This mode focuses on clear market shifts, ignoring flat days. Under these conditions, the model achieved a 61% accuracy, indicating it can catch major directional trends.
- 3-Way (Up, Down, or Flat): This is a much harder challenge that includes days where prices barely move. Even in this scenario, the model maintained a 55.56% accuracy, outperforming a simple random guess.

Testing Methodology. For every prediction, the model was given 5 days of news context to predict the market’s direction on the 6th day. This 5-to-1 ratio ensures the model has enough memory of recent events to make an informed prediction.

Accuracy Calculation. Accuracy is calculated as the percentage of times our model correctly guessed the market’s direction:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Trading Days}} \quad (1)$$

By visualizing the model’s focus, we confirmed it was correctly pinpointing major events like OPEC announcements, proving it wasn’t just guessing, but actually following the logic of the market.

7 Project Limitations and Future Work

While our pipeline demonstrates a robust end-to-end framework, it was constrained by two primary factors: time and data volume.

The 12-Day Constraint. Because our collection period was limited to 12 trading days, the model only saw a tiny snapshot of the global economy. In financial markets, 12 days is rarely enough to capture a full range of behaviors (like a sudden crash vs. a steady bull market). This scarcity of examples meant that if a unique event happened only once, the model had no way to learn its pattern reliably.

High Variance in Results. We observed that the model’s test accuracy was highly sensitive to how we split the data. Changing the cutoff date by even one day could swing accuracy significantly. This high variance is a classic symptom of small datasets; it means that while the model can be very accurate for certain periods, it hasn’t yet reached the stability needed for long-term trading. Future work will focus on expanding this dataset to a 6-month window to smooth out these fluctuations and build a truly weather-proof market predictor.